Learning to See in the Dark – Enhancement: Bridge U-Net, Few-Shot Camera Adaptation and Diffusion Bridges

SYS 5185 Final Project (revised)

Abstract

Images taken in extreme low light with a short exposure are dominated by noise and colour shifts. The SID U-Net, trained on RAW short/long exposure pairs, is a strong baseline, but it is tied to one camera and outputs a regression average. We revisit three ways to improve it, re-implemented from scratch and evaluated on the full-resolution SID Sony test set and on three ELD cameras. (1) A *Bridge U-Net* refines the bottleneck with FiLM blocks conditioned on exposure ratio and noise level. With teacher-latent distillation it is slightly worse than the baseline (28.36 vs. 28.62 dB); without distillation it is the most accurate model (28.94 dB), and FiLM adds 0.25 dB. (2) For new cameras, fine-tuning on six scenes recovers about 3 dB, and five scenes per camera are as good. We fix a leak in the original meta-learning setup, where support and query shared targets, and build scene-disjoint tasks with simulated sensor responses. The meta-learned initialisation is best without adaptation and with short adaptation (+0.9 to +1.2 dB), but a plain SID U-Net adapts best with 200 Adam steps. (3) The feature-space "DDBM" of the first version collapses under its own iterative inference (18.0 dB). A simplified pixel-space DDBM reaches 28.03 dB, below regression on PSNR as theory predicts, but it is 14% better on the perceptual metric LPIPS (0.394 vs. 0.459). The released DDBM recipe learns very slowly with its default noise scale ($\sigma_{\max} = 80$: 24.1 dB on validation after 60k iterations). The full DDBM, with the released VP bridge and 40-step sampler and 303k iterations, has the best LPIPS (0.291, 37% below the U-Net) and the sharpest images, but it falls to 27.35 dB and SSIM 0.696, and part of its detail is invented.

1 Introduction

A phone camera at night faces a hard trade-off. A long exposure collects enough light, but the camera must stay still for seconds and anything that moves blurs. A short exposure freezes motion, but at 1/30 s in a dim room the sensor records only a few photons per pixel. Shot noise, read noise and quantisation then dominate the signal, colours shift, and dark regions lose their structure. Restoring a clean, correctly coloured image from such a frame matters for night photography, surveillance, driving, drones and search-and-rescue. It is also a hard inverse problem, because much of the scene information is simply missing from the input.

The field moved in three steps. Classical pipelines denoise and then tone-map, or merge a burst of frames [3, 6]. Methods that work on the processed sRGB image [5, 10, 29] break down in *extreme* low light, because the camera's image processor has already quantised and amplified the noise. Chen et al. [2] showed that a single U-Net [20] trained on *RAW* short exposures and their long-exposure counterparts (the SID dataset) beats these pipelines by a wide margin. Two problems remained. First, paired data are camera-specific: a SID-trained model degrades on other sensors, and capturing new pairs for every camera is expensive. ELD [30] addresses this with calibrated noise synthesis, and Prabhakar et al. [18] with few-shot adaptation. Second, the regression network outputs a single blurry average when information is missing. Diffusion models [8, 22], and in

particular *diffusion bridges* that diffuse directly between the degraded and the clean image [14, 37], promise sharper and more faithful outputs, at the cost of many network evaluations.

This project builds on the SID U-Net and asks three questions:

1. **Can the U-Net be told what it is looking at?** In extreme low light, noise and content mix most at the bottleneck. We add a *bridge* module there that refines the deepest features before decoding. It is conditioned by FiLM [17] on the exposure ratio and an estimated noise level. During training it is pulled towards the features of the clean long exposure by teacher-latent distillation [19], which costs nothing at test time.

2. **Can it adapt to a new camera from a few scenes?** We compare full fine-tuning on ELD cameras with K-shot adaptation ($K = 1, 3, 5$ scenes) from a supervised and a meta-learned (MAML [4]) initialisation. We redesign the meta-learning tasks so that support and query never share a scene, and we state exactly how many scenes and samples each task uses.

3. **Does a diffusion bridge help?** We compare the feature-space "DDBM-Feature" model of the original project with a real pixel-space Denoising Diffusion Bridge Model [37] trained by bridge score matching, first in a simplified form and then with the bridge, sampler and training length of the released code.

Takeaways. (i) Refining the bottleneck with FiLM blocks conditioned on the degradation improves the SID U-Net by 0.32 dB on the full-resolution SID test set, but only without teacher-latent distillation, which costs 0.58 dB. (ii) One scene of a new camera adds about 2 dB over the zero-shot model on ELD, and five scenes per camera match full fine-tuning. With scene-disjoint tasks, a meta-learned initialisation is the best starting point and adapts best in a few steps (+0.9 to +1.2 dB), but this advantage disappears with a long adaptation. (iii) The feature-space "DDBM" of the first version collapses under iterative inference (18.0 dB). A simplified pixel-space DDBM reaches 28.03 dB, 0.6 dB below regression, which is expected because PSNR rewards the posterior mean; on the perceptual metric LPIPS it is better than the U-Net (0.394 vs. 0.459). The released VE recipe learns very slowly unless its noise scale is matched to the data. The full DDBM of the released code goes furthest in the same direction: the best LPIPS we measured (0.291) and visibly sharper images, at 27.35 dB and 119 network evaluations. Where the input holds no signal, that sharpness is invented detail.

Changes from the first version. The original code of this project was lost. Everything here was re-implemented from the first report, retrained and re-evaluated on full-resolution test images. In the process we found and fixed a flaw in the original meta-learning setup: support and query targets coincided. We also added the SID baseline, ablations and the pixel-space DDBM, which the first report announced but did not include.

2 Related Work and Background

Low-light enhancement on sRGB images. Classical enhancement stretches the histogram or decomposes an image into reflectance and illumination following Retinex theory [12]. Learned methods keep this structure: RetinexNet [29] learns the decomposition, Zero-DCE [5] predicts pixel-wise tone curves without paired data, EnlightenGAN [10] trains adversarially on unpaired images, and general restoration backbones such as Restormer [32] are also used for the task. All of these start from an 8-bit sRGB image. In extreme low light, the camera ISP has already quantised the signal and amplified the noise in that image, so there is little left to recover.

Extreme low-light imaging in the RAW domain. Denoising before the ISP keeps the full bit depth and a simple noise model. Hand-crafted pipelines combine a denoiser such as BM3D [3] with burst alignment and merging [6]. *Learning to See in the Dark* (SID) [2] replaced the whole pipeline with one network: a U-Net maps the packed Bayer RAW of a short exposure, multiplied by the exposure ratio, directly to the sRGB rendering of a long exposure. The authors released the paired SID dataset that we use. Because paired data are expensive to capture for each sensor, later work synthesises training pairs from a physics-based noise model calibrated per camera (ELD [30, 31]; see also [34]). ELD also released a real test set of four cameras, which we use as an unseen-sensor benchmark. Prabhakar et al. [18] showed that a SID-trained network adapts to a new camera from only a few paired images. That is exactly the question of our few-shot experiment.

U-Net. The U-Net [20] is an encoder–decoder. The encoder halves the resolution four times while doubling the channels (32 to 512 in SID). The decoder upsamples back, and *skip connections* concatenate each encoder level with the decoder level of the same size, so fine spatial detail lost in down-sampling is restored. The deepest level (the *bottleneck*) has the largest receptive field and carries global information such as illumination and colour cast. In extreme low light it is also where the shot and read noise of the input are most entangled with content. Our bridge module operates there.

Feature-wise conditioning and distillation. FiLM [17] conditions a network on side information by predicting a per-channel scale γ and shift β from a condition vector and applying $\gamma \odot h + \beta$ to an intermediate feature map h. It is a cheap way to make one network behave differently for different degradation levels. Knowledge distillation [7] trains a student to match a teacher's outputs. FitNets [19] match intermediate *features* instead ("hints"). When the teacher sees information unavailable at test time, this is learning using privileged information [26]. In our case the privileged input is the long exposure.

Diffusion and diffusion bridge models. Denoising diffusion models [8, 25] learn to reverse a process that gradually turns data into Gaussian noise. Image-to-image tasks condition the reverse process on the degraded input (SR3 [22], Palette [21]), but sampling still starts from pure noise. *Bridge* models instead diffuse between the two images of a pair. A Brownian bridge pinned at the clean image x_0 at $t = 0$ and the degraded image x_T at $t = T$ has the marginal

$$x_t \mid x_0, x_T \sim \mathcal{N}\Big((1 - \tfrac{t}{T})x_0 + \tfrac{t}{T}x_T,\ \sigma^2\, t\,(1 - \tfrac{t}{T})\, I\Big), \tag{1}$$

so sampling starts from the degraded image rather than from noise. BBDM [13] and I^{2}SB [14] use this construction. *Denoising Diffusion Bridge Models* (DDBM) [37] generalise it with Doob's h-transform. They learn the score of the bridge by denoising bridge score matching (equivalently, by predicting x_0 from x_t, t and x_T) and sample with a reverse SDE or ODE. Diffusion Bridge Implicit Models [36] give a DDIM-like [24] sampler for the same trained network that works with few steps. The original version of this project used only a deterministic, feature-space imitation of DDBM. Here we also train the real pixel-space model (Section 3.6).

Meta-learning. Few-shot learning asks a model to generalise from a handful of labelled examples [16]. Meta-learning ("learning to learn") [9] trains on a *distribution of tasks* so that a new task is learned quickly. MAML [4] looks for an initialisation θ from which a few gradient steps on

a task's *support set* give low loss on the same task's *query set*:

$$\min_{\theta} \sum_{\tau} \mathcal{L}_{\tau}^{\text{query}}\Big(\theta - \alpha \nabla_{\theta} \mathcal{L}_{\tau}^{\text{support}}(\theta)\Big)\,. \tag{2}$$

Back-propagating through the inner step needs second derivatives. First-order MAML and Reptile [15] drop them. In image restoration, MZSR [23] meta-learns a super-resolution network that adapts to an unseen blur kernel in one step. A task must contain a support set and a disjoint query set drawn from the same distribution. How tasks are built therefore matters as much as the algorithm (Section 3.5).

Evaluation metrics. We report PSNR, SSIM [28] and MS-SSIM [27]. MS-SSIM compares luminance, contrast and structure over five dyadic scales. It agrees better with perception than PSNR and is also a useful training loss [35]. LPIPS [33] instead measures the distance between two images in the features of a trained network and tracks human similarity judgements more closely. Blau and Michaeli [1] prove that low distortion and high perceptual quality, the latter defined as closeness to the distribution of natural images, cannot both be reached beyond a certain point. LPIPS compares an output to its target rather than two distributions, so we use it as a proxy for perceptual quality.

3 Method

3.1 Data and pre-processing

SID Sony. The Sony subset of SID [2] has 231 indoor and outdoor scenes shot with a Sony α7S II (Bayer CFA, 14-bit, 4256×2848). Each scene has one long exposure (10 s or 30 s), which serves as ground truth. It also has several short exposures of 1/10, 1/25 or 1/30 s, i.e. exposure ratios of $\times$100, $\times$250 and $\times$300. We use SID's own split by scene id: 161 training scenes (1865 short/long pairs), 20 validation scenes (234 pairs) and 50 test scenes (598 pairs). A training scene has between 8 and 22 short exposures. All short exposures of a scene share the *same* long-exposure target.

ELD. To test generalisation to new sensors we use the real ELD test set [30]. It has 10 scenes per camera, and we use the three Bayer cameras of the original project: Canon EOS 70D, Canon EOS 700D and Nikon D850. In every scene, images 1, 6, 11 and 16 are long-exposure references at ISO 800, 1600 and 3200. The four frames after each reference are short exposures at ratios $\times$1, $\times$10, $\times$100 and $\times$200. We take scenes 1–6 of each camera as the pool for adaptation and fine-tuning, and scenes 7–10 as a held-out test set. We evaluate on the $\times$100 and $\times$200 frames as in [30], which gives 24 test pairs per camera. (The original project held out one scene per camera, which is too small to measure differences between methods.)

Packing and normalisation. Each RAW frame is subtracted by its black level, divided by (white $-$ black) and packed into four half-resolution planes in R, G_1, B, G_2 order. We read the CFA layout from the file, so Canon and Nikon frames are packed consistently. The short exposure is multiplied by the exposure ratio (clipped at 300) and clipped to $[0, 1]$. The target is the long exposure rendered by LibRaw with the camera white balance and no auto-brightening, as in SID. All arrays are cached once as 16-bit files and memory-mapped, so a training crop reads only its own pixels. Training uses aligned random 512×512 packed crops (1024×1024 target), random flips and transposition.

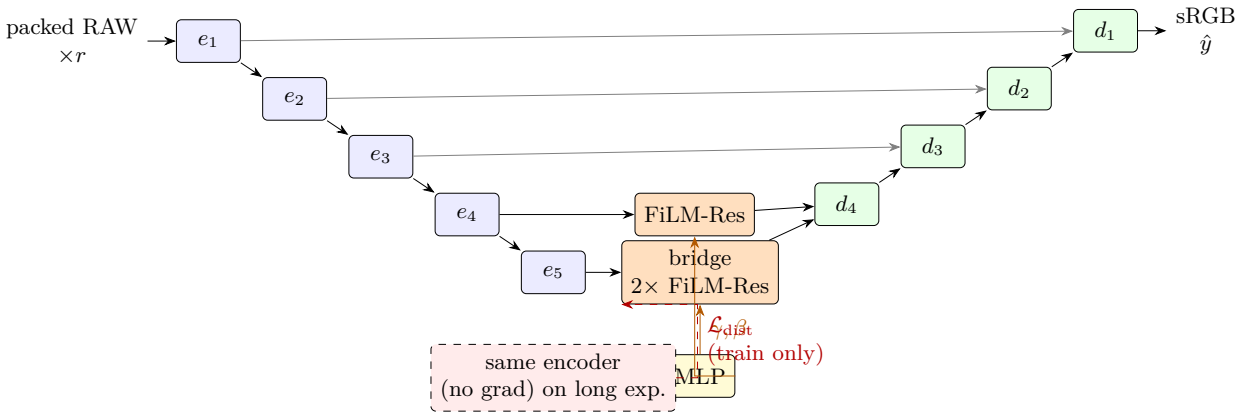


Figure 1: Bridge U-Net. The SID encoder/decoder are unchanged. FiLM residual blocks refine the bottleneck e_5 and the skip e_4, conditioned on exposure ratio and estimated noise level. During training, features of the long exposure from the same encoder supervise the refined features (teacher-latent distillation).

3.2 Baseline: SID U-Net

The baseline is the SID network: a 5-level U-Net with $\{32, 64, 128, 256, 512\}$ channels, two 3×3 convolutions with LeakyReLU per level, max-pool down-sampling, transposed-convolution up-sampling and concatenating skips. A final 1×1 convolution produces 12 channels, which a depth-to-space layer turns into the full-resolution RGB image (7.8M parameters).

3.3 Bridge U-Net

The Bridge U-Net keeps the encoder and decoder of the baseline and inserts a *bridge* between them (Fig. 1). Its three parts address three weaknesses of the plain U-Net in this setting.

Bottleneck refinement. The bottleneck feature e_5 passes through $n_b = 2$ residual blocks $e_5 \leftarrow e_5 + W_2\,\phi(\mathrm{FiLM}(W_1 e_5))$. W_2 is zero-initialised, so the network starts exactly as the baseline. The 4th-level skip feature e_4 passes through one more such block before it reaches the decoder ("conditional skip refinement"). The plain U-Net instead copies the noisy e_4 straight into the decoder.

FiLM conditioning on the degradation. The same network must handle $\times 100$ to $\times 300$ amplification and very different noise. We give it a two-number condition: the normalised log exposure ratio $\log r / \log 300$ and the log noise level $\hat{\sigma}$. $\hat{\sigma}$ is estimated from the input as the median absolute deviation of $G_1 - G_2$, which removes scene content. An MLP maps the condition to a 64-d embedding, and each bridge block predicts its own (γ, β) from it [17].

Teacher-latent distillation. During training only, the long-exposure RAW (packed and normalised the same way) goes through the *same* encoder without gradient. Its features t_4, t_5 are what the encoder would produce from a clean input. We pull the refined student features towards them [19, 26]:

$$\mathcal{L}_{\mathrm{dist}} = \tfrac{1}{2} \sum_{l \in \{4,5\}} \frac{1}{|\Omega_l|} \sum_{p \in \Omega_l} \Big(1 - \frac{\langle e_l(p), t_l(p) \rangle}{\|e_l(p)\|\,\|t_l(p)\|}\Big), \tag{3}$$

the mean cosine distance between student and teacher feature vectors over all pixels p. The loss compares directions and not magnitudes, which differ between an amplified short exposure and a long exposure. (The first version used an L1 distance between normalised vectors, which is roughly $1/\sqrt{C}$ smaller per element and was negligible next to the image loss.) The full loss is $\mathcal{L} = \|\hat{y} - y\|_1 + 0.1\,(1 - \text{MS-SSIM}(\hat{y}, y))\,[\text{step} \geq s_0] + \lambda \mathcal{L}_{\text{dist}}$ with $\lambda = 0.1$. MS-SSIM is switched on after s_0 steps, once the image is roughly right. At test time the long exposure is not needed. The model has 18.55M parameters.

3.4 Fine-tuning to new cameras

To move a SID-trained model to the ELD cameras, we fine-tune all layers on every pair of scenes 1–6 of the three cameras (216 pairs, all four ratios). We use AdamW (lr 5×10^{-5}), cosine decay, gradient clipping at 1, an EMA of the weights (decay 0.999) and the same loss as above. The variant *decoder+bridge* freezes the encoder. Model selection uses the held-out scenes 7–10.

3.5 Few-shot adaptation with meta-learning

Question. Full fine-tuning uses 6 scenes per camera. Can a model adapt to a new sensor from $K \in \{1, 3, 5\}$ scenes, and does meta-learning give a better starting point for that than ordinary supervised pre-training?

What went wrong in the first version. The original project treated each SID *scene* as a task, with 5 short exposures as support and up to 8 more from the same scene as query. Because all short exposures of a scene share one long-exposure target (Section 3.1), the query targets were already in the support set. The inner loop could memorise the answer, so the meta-objective did not measure adaptation. This explains why the curves in the original report stayed flat.

Task construction. A task must differ from other tasks in something the model can learn from a few examples. It must also have a query that is *not* in its support. On a new camera, what changes is mainly the colour response of the sensor (and its noise). We therefore simulate a new sensor per task:

- a sensor response $g = (g_R, g_G, g_B, g_G)$ with $\log g_c \sim \mathcal{U}(-0.35, 0.35)$, multiplied into the packed RAW of both short and long exposures, while the sRGB target is left unchanged. The model must learn to undo an unknown colour transform;
- one exposure ratio $r \in \{100, 250, 300\}$;
- a support set of $K = 5$ *scenes* and a query set of $Q = 8$ *other* scenes, all drawn from the 161 training scenes, with one random short exposure of ratio r per scene and a random 256×256 packed crop.

The number of distinct tasks is effectively unlimited: $\binom{161}{5}\binom{161-5}{8}$ scene choices times a continuous gain space. We meta-train for 2,000 outer steps with 2 tasks each, i.e. 4,000 sampled tasks covering every training scene many times. In the inner loop we take 3 SGD steps with $\alpha = 10^{-3}$ on the support L1 loss. In the outer loop we use Adam (lr 10^{-5}) on the query L1 loss through the adapted weights (second-order MAML, Eq. 2), starting from the trained Bridge U-Net.

Meta-test. The real test tasks are the three ELD cameras. For each camera and $K \in \{0, 1, 3, 5\}$, the support set is all pairs of scenes $1..K$ (12 pairs per scene). We adapt with 200 Adam steps (lr 2×10^{-5}, batch 4, 512 crops) and evaluate on the 24 test pairs of scenes 7–10, so support and query never share a scene. We compare three initialisations under the same adaptation: the SID U-Net, the Bridge U-Net and the meta-learned Bridge U-Net.

3.6 Diffusion bridge models

DDBM-Feature (the original variant). The original project kept the Brownian bridge of Eq. 1, but in feature space. For levels 3–5 of the encoder, the decoder receives $f_t = (1-t)f_{\text{short}} + tf_{\text{long}} + \sigma_{\max}\sqrt{t(1-t)}\,\epsilon$ with $\sigma_{\max} = 0.1$ and $t \sim \mathcal{U}(0.01, 0.99)$. A sinusoidal embedding of t modulates every decoder block, and the network predicts the clean image directly with $\mathcal{L}_1 + 0.1(1 -$ MS-SSIM). At test time no long exposure exists. The output y_{k-1} is mosaicked back to a pseudo-RAW, re-encoded as f_{long} and decoded at $t_k = k/N$, for $N = 3$ deterministic steps. No score is learned and the sampler is not a reverse diffusion, so this is a time-conditioned U-Net with feature mixing as augmentation rather than a DDBM.

DDBM (pixel space, new). We also implement the actual model of [37] with a VE/Brownian-bridge schedule. The endpoints are the target $x_0 = 2y - 1$ and a *naive rendering* of the input, x_T: the amplified packed RAW with averaged greens, camera white balance, gamma 1/2.2 and bilinear $2\times$ up-sampling, in $[-1, 1]$. With $T = 1$ and $\sigma^2 = 0.5$, Eq. 1 gives x_t. A time-conditioned U-Net $D_\theta(x_t, t, x_T, \text{RAW})$ (pixel-unshuffled inputs, channels $\{64, \ldots, 512\}$, 15.73M parameters) predicts x_0. Its loss $\|D_\theta - x_0\|^2$ is denoising bridge score matching in the x_0-parameterisation, since the bridge score is $((1-t)D_\theta + tx_T - x_t)/(\sigma^2 t(1-t))$. To sample, we walk t from 1 to 0 in N steps. Each step draws from the exact posterior of a Brownian bridge pinned at $\hat{x}_0 = D_\theta$ and the current state:

$$x_s \mid x_t, \hat{x}_0 \sim \mathcal{N}\Big((1 - \tfrac{s}{t})\hat{x}_0 + \tfrac{s}{t}x_t,\ \sigma^2 \tfrac{s(t-s)}{t} I\Big), \tag{4}$$

which is the ancestral sampler ($\eta = 1$). The deterministic DBIM sampler [36] ($\eta = 0$) reuses the predicted noise instead. Table 1 contrasts all bridge variants of this report.

DDBM with the released recipe. The model above is a simplified DDBM: a small bridge variance, an unweighted x_0 loss and a first-order sampler. To check that these simplifications do not explain our findings, we also train the recipe of the released DDBM code [37], which follows EDM [11]. The bridge is a VE bridge indexed by the noise level $\sigma \in [\sigma_{\min}, \sigma_{\max}]$, with marginal

$$x_\sigma = a\,x_T + (1-a)\,x_0 + \sigma\sqrt{1-a}\;\epsilon, \qquad a = \sigma^2/\sigma^2_{\max}. \tag{5}$$

The network is wrapped in the EDM preconditioning

$$D_\theta = c_{\text{skip}}(\sigma)\,x_\sigma + c_{\text{out}}(\sigma)\,F_\theta(c_{\text{in}}(\sigma)x_\sigma,\ \tfrac{1}{4}\log\sigma,\ x_T, \text{RAW}), \tag{6}$$

where $c_{\text{in}}, c_{\text{skip}}, c_{\text{out}}$ give unit-variance inputs and targets for $\sigma_{\text{data}} = 0.5$. The loss is weighted by $\lambda(\sigma) = 1/c_{\text{out}}(\sigma)^2$ and $\sigma \sim \mathcal{U}(\sigma_{\min}, \sigma_{\max})$. Sampling uses the hybrid sampler of DDBM on a Karras schedule ($\rho = 7$): each step is a stochastic Euler–Maruyama step of the bridge SDE over a fraction 0.33 of the interval, followed by a Heun step of the probability-flow ODE, whose drift contains the Doob h-transform term $\nabla \log p(x_T \mid x_\sigma) = (x_T - x_\sigma)/(\sigma^2_{\max} - \sigma^2)$. N steps cost $3N - 1$ network evaluations. We use the same network F_θ as above and train two VE versions for 60k steps each: the released default $\sigma_{\max} = 80$ and $\sigma_{\max} = 1$, which matches the noise scale to the $[-1, 1]$ data range. Everything else is identical.

Table 1: The diffusion-bridge variants compared in this report.

	DDBM-Feature	DDBM (simplified)	DDBM (released recipe)	DDBM (full)
space	encoder features	pixels (sRGB)	pixels (sRGB)	pixels (sRGB)
endpoints	short / long features	x_T = naive render, x_0 = GT	same	same
bridge	BB, $\sigma_{\max} = 0.1$	BB, $\sigma^2 = 0.5$	VE, $\sigma_{\max} \in \{80, 1\}$	VP, $\beta_d = 2$, $\beta_{\min} = 0.1$
network output	image	x_0	EDM-preconditioned x_0	same
loss	$\mathcal{L}_1$ + MS-SSIM	$\|D - x_0\|^2$	$\lambda(\sigma)\|D - x_0\|^2$	same
score learned	no	yes	yes	yes
sampler	re-encode, $N = 3$	bridge posterior / DBIM	Euler–Maruyama + Heun	same, D_θ clipped
NFE for N steps	$N + 1$	N	$3N - 1$	$3N - 1$
training iterations	40k	60k	60k	303k

The full DDBM. The two VE runs still differ from the released code in the bridge, the sampler settings and the training length. Our last DDBM follows the release in all three. It uses the VP bridge of DDBM, the variant with the lower FID in the image-translation experiments of [37]. With $\alpha_t = e^{-\beta_d t^2/4 - \beta_{\min} t/2}$, $\sigma_t^2 = 1 - \alpha_t^2$ and $\mathrm{SNR}_t = \alpha_t^2/\sigma_t^2$, its marginal is

$$x_t = \frac{\mathrm{SNR}_T}{\mathrm{SNR}_t}\frac{\alpha_t}{\alpha_T}\, x_T + \alpha_t\Big(1 - \frac{\mathrm{SNR}_T}{\mathrm{SNR}_t}\Big)x_0 + \sigma_t\sqrt{1 - \frac{\mathrm{SNR}_T}{\mathrm{SNR}_t}}\;\epsilon, \tag{7}$$

with the released $\beta_d = 2$, $\beta_{\min} = 0.1$ and $t \in [10^{-4}, 1]$. The bridge noise peaks at a standard deviation of 0.52, so this bridge is matched to the data range by construction. The preconditioning, the loss weight $1/c_{\text{out}}^2$ and $t \sim \mathcal{U}(t_{\min}, T)$ are as for the VE bridge, with $c_{\text{in}}, c_{\text{skip}}, c_{\text{out}}$ derived from the coefficients above. We sample with the released defaults: 40 steps of the hybrid sampler (119 NFE), churn 0.33, guidance weight 1 on the h-transform term, and D_θ clipped to $[-1, 1]$ at every evaluation. The weights are averaged with an EMA of decay 0.9999 as in the release, with the usual warm-up $\min(0.9999, \frac{1+n}{10+n})$ so that early validation scores are meaningful. We train for 5.5 h, i.e. 303k iterations, five times longer than the other pixel-space DDBMs. Before training we checked all three bridges with an oracle network that returns the true x_0: every sampler then carries x_T to x_0 (RMSE below 2×10^{-3}). The one part we do not reproduce is the network. The released models use the ADM U-Net with self-attention. At full SID resolution, attention would cover tens of thousands of positions in each of the 119 evaluations, so all our DDBMs share the convolutional U-Net F_θ.

4 Experiments

4.1 Setup

All models were trained and evaluated on one NVIDIA A100 on Google Colab, with bf16 mixed precision. The supervised models (SID U-Net, Bridge U-Net and its ablations, DDBM-Feature) use AdamW with learning rate 2×10^{-4}, a 500-step warm-up, cosine decay to 2%, batch 8, 512×512 packed crops and 40k iterations. MS-SSIM enters the loss after 20k iterations. Every 2k iterations we evaluate an EMA of the weights (decay 0.999) on 60 validation pairs (centre 512×512 packed crop), and we test the best validation checkpoint. The pixel-space DDBMs use learning rate 10^{-4} (1k-step warm-up, then constant), batch 16, 256×256 crops and are validated on 20 pairs with $N = 10$ steps. Besides the fidelity metrics PSNR, SSIM and MS-SSIM we report LPIPS [33] (AlexNet, lower is better) on the full-resolution images, a learned perceptual distance that, unlike PSNR, does not reward a blurry average. Unless stated otherwise, **test numbers are on all 598 SID test pairs at full resolution** (4256×2848), not on crops. Each configuration was trained

Table 2: SID Sony test set (598 full-resolution pairs). NFE: network evaluations per image; time: seconds per full-resolution image on an A100 (network only). LPIPS was added after the first set of checkpoints was lost, so it is measured on retrained copies (†: same configuration and seed; the copies score 28.61, 27.97 and 27.88 dB PSNR, within 0.06 dB of the original runs). The best value per column is in bold.

Model	Params	NFE	PSNR↑	SSIM↑	MS-SSIM↑	LPIPS↓	Time (s)
SID U-Net, reported in [2]	7.8M	1	28.88	0.787	–	–	–
SID U-Net (ours)	7.76M	1	28.62	0.796	**0.876**	0.459†	0.024
SID U-Net (ours, second seed)	7.76M	1	28.61	0.796	0.876	–	
Bridge U-Net (FiLM + distillation)	18.55M	1	28.36	0.789	0.868	–	0.024
w/o distillation	18.55M	1	**28.94**	**0.798**	**0.876**	–	0.024
w/o FiLM	18.55M	1	28.11	0.787	0.868	–	0.024
DDBM-Feature, iterative ($N = 3$)	7.92M	4	18.02	0.449	0.742	–	0.098
DDBM-Feature, single pass	7.92M	1	25.51	0.733	0.841	–	0.025
DDBM, simplified, $N = 5$	15.73M	5	28.03	0.782	0.862	0.394†	0.257
DDBM, simplified, $N = 10$	15.73M	10	27.90	0.779	0.856	0.394†	0.514
DDBM, simplified, $N = 10$, $\eta = 0$	15.73M	10	27.88	0.776	0.854	–	0.514
DDBM, simplified, $N = 20$	15.73M	20	27.77	0.776	0.851	–	
DDBM, released recipe, $\sigma_{\max} = 1$, $N = 5$	15.73M	14	23.59	0.406	0.575	0.641	0.741
DDBM, released recipe, $\sigma_{\max} = 1$, $N = 10$	15.73M	29	25.57	0.585	0.721	0.465	1.533
DDBM, released recipe, $\sigma_{\max} = 1$, $N = 20$	15.73M	59	26.24	0.641	0.762	0.397	
DDBM, full (VP bridge), $N = 20$	15.73M	59	26.97	0.675	0.787	0.316	3.560
DDBM, full (VP bridge), $N = 40$	15.73M	119	27.35	0.696	0.804	**0.291**	7.188

once (one seed). To estimate run-to-run variation we retrained the SID U-Net with a second seed (Section 4.2).

One practical note: the first Bridge U-Net run diverged after 2.5k iterations (the loss jumped from 0.05 to 0.4 and never recovered). We restarted it and, for all runs, skip a batch whose loss exceeds five times the running average after the first 1k iterations (at most 10 in a row), which happened only a handful of times.

4.2 SID: baseline, Bridge U-Net and ablations

Table 2 gives the main results. Our SID U-Net reaches 28.62 dB, 0.26 dB below the original paper [2] and with a higher SSIM. The paper trains for 4000 epochs and we train for 40k iterations of batch 8, so the reproduction is faithful. A second training run with a different seed gives 28.61 dB (SSIM 0.796), a difference of only 0.01 dB on the test set (33.07 vs. 32.89 dB on validation). Two runs are a weak estimate of the variance, but they suggest that the test-set gaps of 0.25 dB and more discussed below are not run-to-run noise.

The bridge module. With both FiLM and distillation, the Bridge U-Net (28.36 dB) is slightly *worse* than the baseline. The ablations show why. Removing FiLM costs a further 0.25 dB, so conditioning on exposure ratio and noise level helps. Removing distillation *gains* 0.58 dB, and the Bridge U-Net without distillation is the most accurate model of this report: 28.94 dB, +0.32 dB over the baseline. The same order holds on the validation set (33.17 vs. 33.07 dB for SID and 32.35 dB with distillation). The distillation loss itself stays small, around 3×10^{-4} in cosine distance: the encoder already maps the amplified short exposure and the long exposure to features of nearly the same direction. The loss therefore does little to clean the features. What it does do is pull the output of the bridge blocks back towards the plain encoder features, which removes most of the

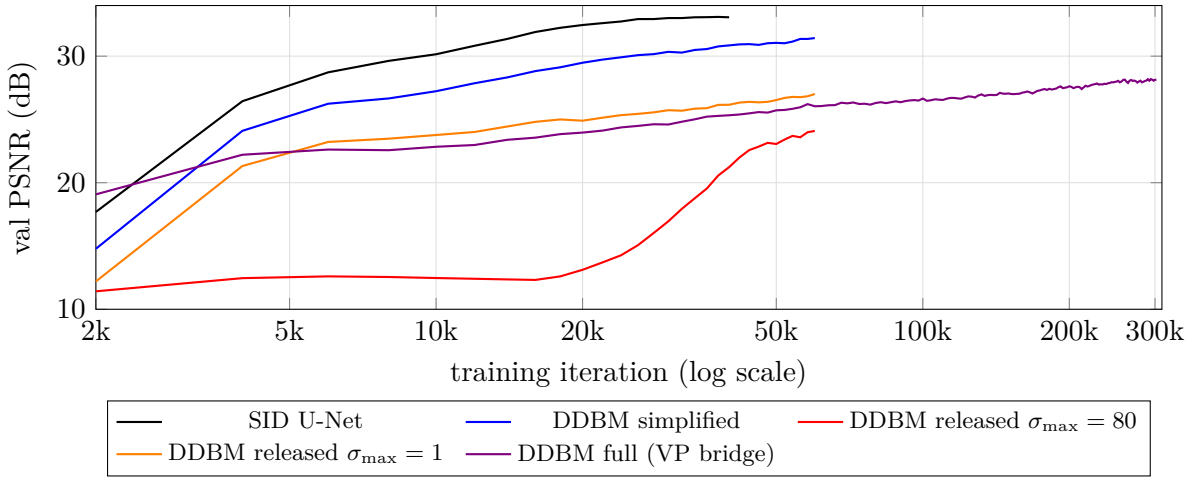


Figure 2: Validation PSNR during training (20–60 SID validation pairs, centre crops, $N = 10$ sampling steps for the DDBMs; log-scale iterations). With the released noise scale $\sigma_{\max} = 80$ the DDBM learns very slowly. Matching the noise scale to the data ($\sigma_{\max} = 1$) speeds up training a lot, but after 60k iterations the recipe still trails the simplified DDBM by about 4 dB (27.0 vs. 31.2–31.4 dB for its two runs). The full DDBM with the VP bridge leads at 2k iterations, then flattens and is at 26.04 dB after 60k iterations. Five times longer training brings it to 28.19 dB, still about 3 dB below the simplified DDBM at 60k, and its last 100k iterations add only 0.5 dB.

freedom the bridge was added for. In our setting, teacher-latent distillation is a regulariser that hurts.

4.3 Diffusion bridges

DDBM-Feature does not work as a diffusion model. The feature-space variant of the original project reaches 25.51 dB when it is run as a single pass at $t = 0$, i.e. as a time-conditioned U-Net. Its intended iterative inference, which re-encodes its own output as the "long exposure" feature, collapses to 18.02 dB. During training the decoder always sees features of the *true* long exposure, while at test time it sees features of its own previous estimate, so errors compound from step to step. The longer we train, the more the decoder relies on these features: the validation PSNR of the iterative sampler peaks at 19.7 dB after 6k iterations and falls to 12.3 dB at 40k. A bridge whose endpoint is unavailable at test time cannot be sampled this way. This is why we implemented the pixel-space model, whose endpoint x_T comes from the input itself.

A real DDBM is competitive but does not beat regression on PSNR. The simplified pixel-space DDBM reaches 28.03 dB with $N = 5$ steps, 0.6 dB below the SID U-Net, with twice the parameters and five times the compute. More sampling steps make it *worse*: 27.90 dB at $N = 10$ and 27.77 dB at $N = 20$. This is expected. PSNR and SSIM reward the posterior mean, which is exactly what an L_1/L_2 regression network estimates. A diffusion model is trained to draw *samples* from the posterior, and for an exact sampler the expected squared error of a sample is twice that of the posterior mean. Fewer steps behave more like a single denoising step and stay closer to the mean, so they score higher. The deterministic DBIM sampler ($\eta = 0$) gives the same PSNR as the ancestral one. The possible benefit of diffusion, sharper texture instead of a blurry average, is not what these metrics measure.

On a perceptual metric the DDBM is better. LPIPS reverses the order. The simplified DDBM scores 0.394 against 0.459 for the SID U-Net, 14% lower, while being 0.6 dB worse in PSNR. This is the perception–distortion trade-off [1]: no estimator can be best on both, and the regression network sits at the low-distortion end. Unlike PSNR, LPIPS does not get worse with more steps ($N = 5$ and $N = 10$ give the same value), so the extra steps move the output away from the mean without making it look less like the target. The released recipe shows the same effect more strongly. From $N = 10$ to $N = 20$ its LPIPS improves from 0.465 to 0.397, almost reaching the simplified model, although its PSNR (26.24 dB) stays 1.8 dB lower. The gain is modest in the crops of Fig. 3: the simplified DDBM looks much like the U-Net, slightly less smooth in flat regions, and in the $\times 250$ scene its colours drift towards yellow. Because LPIPS was measured on retrained copies, this comparison rests on one run per model.

The released recipe: the noise scale explains the slow start, not the gap. Trained with its default $\sigma_{\max} = 80$ for 60k iterations, the released DDBM recipe reached only 24.09 dB on validation (SSIM 0.405), against 31.43 dB for the simplified DDBM with the same network (Fig. 2). Its validation PSNR stayed at about 12 dB for the first 20k iterations. We traced this to the noise scale. Our images lie in $[-1, 1]$, while the bridge noise at $\sigma = \sigma_{\max}/\sqrt{2}$ has a standard deviation of about 40, so in the middle of the bridge x_σ is almost pure noise. And because $\sigma \sim \mathcal{U}(\sigma_{\min}, \sigma_{\max})$, 99% of the training samples have $\sigma > 0.8$, so the low-noise range that decides the final detail is rarely trained. These settings come from image-to-image translation, where the two endpoints differ a lot. With $\sigma_{\max} = 1$ and everything else unchanged, training is much faster: 21.4 dB after 4k iterations and 25.85 dB after 30k, against 12.5 and 17.0 dB for $\sigma_{\max} = 80$ at the same points. We then retrained it for the full 60k iterations, where it reaches 27.01 dB on validation, about 3 dB above $\sigma_{\max} = 80$ at the same point. The test score is still far below the simplified DDBM: 23.59, 25.57 and 26.24 dB with $N = 5$, 10 and 20, with low SSIM (0.406–0.641). Two observations point to the sampler rather than the network. First, more steps help here, the opposite of the simplified model, and the gain has not levelled off at $N = 20$. Second, the outputs keep visible residual noise and invented texture, and the $\times 300$ scene comes out darker than the target (Fig. 3). With the stochastic churn step and the Karras schedule, a handful of steps does not remove the injected noise, while the simplified model's bridge posterior lands exactly on $\hat{x}_0$ in its last step. The test evaluation of the $\sigma_{\max} = 80$ model was lost when the cloud machine was reclaimed, so we report its validation score only.

The full DDBM wins on LPIPS and loses on fidelity. With the released VP bridge, the released sampler and 303k iterations, the DDBM has the best LPIPS in Table 2: 0.291 at $N = 40$, 37% below the SID U-Net and 26% below the simplified DDBM. Against the U-Net it is lower on 45 of the 50 test scenes and at every exposure ratio (0.206 vs. 0.326 at $\times 100$, 0.287 vs. 0.595 at $\times 250$, 0.359 vs. 0.458 at $\times 300$). Fidelity moves the other way. PSNR is 27.35 dB, 1.3 dB below the U-Net and lower on 43 of the 50 scenes, and SSIM falls from 0.796 to 0.696, lower on all 50. This is the trade-off of the simplified DDBM again, only larger, and the PSNR loss is within what theory allows: a perfect posterior sampler doubles the squared error of the posterior mean, which costs 3 dB. The gap to the simplified DDBM is also much smaller on test than on validation (about 0.6 vs. 3 dB), in part because validation uses only $N = 10$ steps.

Figure 4 shows what these numbers mean. At 40 steps the residual noise of the $\sigma_{\max} = 1$ model is gone, in line with the sampler explanation above, although the bridge and the training length changed as well. Where the input still carries signal, the full DDBM restores detail that the U-Net blurs, such as the fence wires at $\times 100$. Where it does not, the detail is invented: at $\times 300$ the



Figure 3: SID test crops (1024 × 768 from the image centre) at exposure ratios ×300, ×250 and ×100 (top to bottom), from the models retrained after the first set of checkpoints was lost; the simplified and $\sigma_{\max} = 1$ DDBMs use $N = 10$ steps, the full DDBM $N = 40$. The input is the amplified short exposure with a naive demosaic and white balance. The SID U-Net is smooth but faithful. The simplified DDBM is close to it, with a yellow colour drift in the middle row. The DDBM with the released recipe and $\sigma_{\max} = 1$ keeps residual noise, invents texture (the book cover) and is darker than the target in the top row. The full DDBM has no visible residual noise and is the sharpest, but it still invents texture and its colours are patchy.

Table 3: ELD test scenes 7–10 (×100 and ×200, 24 pairs per camera, 1024 × 1024 packed crops). PSNR / SSIM, starting from the SID-trained Bridge U-Net, which scores 24.37 dB on average without adaptation. Fine-tuning uses all pairs of scenes 1–6 of the three cameras.

	Canon 70D	Canon 700D	Nikon D850	Mean
Decoder + bridge	**26.16** / 0.740	25.99 / 0.709	29.35 / 0.811	27.17 / 0.753
All layers	26.14 / **0.745**	**26.01** / **0.713**	**29.81** / **0.827**	**27.32** / **0.762**

book title is replaced by sharp strokes that are not letters, and the orange cover turns grey-brown with a made-up texture. The wires also bend in places and colours are patchy (Fig. 3). On the six crops we inspected, the mean luminance gradient relative to the target is 0.54 for the U-Net, 1.2 for the full DDBM and 2.3 for the $\sigma_{\max} = 1$ model at $N = 10$, so the full DDBM comes closest to the sharpness of the target but overshoots it. LPIPS rewards this sharpness, while SSIM punishes structure that is not there.

More steps help on every metric, the opposite of the simplified DDBM: from $N = 20$ to $N = 40$ PSNR rises by 0.37 dB and LPIPS falls from 0.316 to 0.291, so the released sampler has not converged at 20 steps. The price is 119 network evaluations and 7.188 s per image, about 300 times the U-Net. The simplified DDBM stays ahead on fidelity with 5 evaluations, so a small-noise bridge that ends on the denoised estimate (I^2SB [14], our simplified model) remains the cheaper choice when fidelity matters. Which output is better depends on the use. For night photography the sharper image may well be preferred, but for surveillance or driving an invented detail is a risk, and the blur of the regression network is the safer failure.

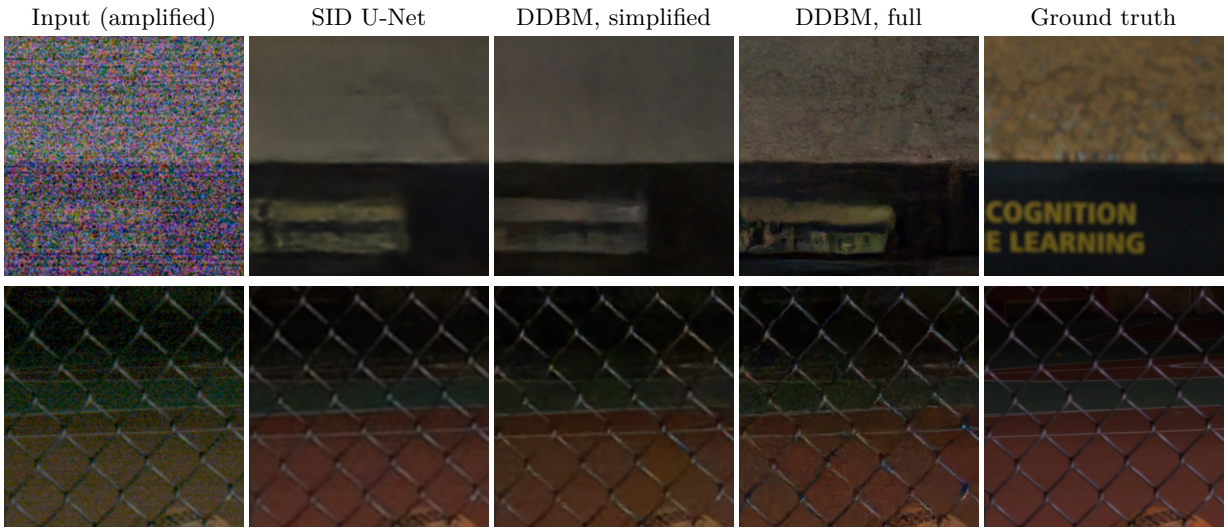


Figure 4: Detail at full resolution (340×340 pixels) from the top and bottom rows of Fig. 3. At $\times 300$ (top) the book title is lost in the input: the U-Net leaves a soft smear and the full DDBM paints sharp strokes that are not letters. At $\times 100$ (bottom) the full DDBM recovers wires that the U-Net blurs.

Table 4: Few-shot adaptation on ELD: PSNR on scenes 7–10, averaged over the three cameras. K adaptation scenes per camera, 12 pairs per scene. *Adam*: 200 Adam steps (lr 2×10^{-5}); *inner*: 30 SGD steps with the meta-training inner learning rate 10^{-3}.

Initialisation	Adaptation	$K = 0$	$K = 1$	$K = 3$	$K = 5$
SID U-Net	Adam	24.17	**26.74**	**27.36**	**27.54**
Bridge U-Net	Adam	24.37	26.31	26.66	27.05
Meta-learned Bridge	Adam	**24.77**	26.48	26.86	27.15
Bridge U-Net	inner	–	24.40	24.53	24.25
Meta-learned Bridge	inner	–	**25.29**	**25.26**	**25.46**

4.4 Adapting to new cameras

Fine-tuning. Without adaptation, the SID-trained model loses more than 4 dB on the ELD cameras (24.37 dB on average, against 28.4 dB on SID). The other sensors have different colour responses and noise, and their frames span ratios down to $\times 1$. Fine-tuning on the 216 pairs of scenes 1–6 recovers about 3 dB (Table 3): 27.32 dB on average for all layers and 27.17 dB with a frozen encoder. Freezing the encoder is almost as good on the two Canons and 0.46 dB worse on the Nikon D850. Most of the camera-specific change can thus be absorbed by the decoder and the bridge, but low-level features still matter for a sensor that differs more from the Sony.

Few-shot adaptation and meta-learning. Table 4 answers the second question. One scene of a new camera (12 pairs) already gives +1.7 to +2.6 dB. Five scenes per camera (27.05–27.54 dB) are as good as fine-tuning on six scenes of all three cameras (27.32 dB). The meta-learned initialisation helps in two cases. It is the best model *before* any adaptation ($K = 0$: +0.40 dB over the Bridge U-Net it started from and +0.60 dB over SID), because meta-training on random sensor gains makes it robust to colour shifts. It is also clearly better when adaptation is as short as in meta-training (*inner*: +0.9 to +1.2 dB at every K). During meta-training the query loss after adaptation (L1

0.0459) was below the support loss before adaptation (0.0510), so the inner loop learned to adapt. With a long adaptation of 200 Adam steps, however, the advantage over the Bridge U-Net shrinks to 0.1–0.2 dB, and the plain SID U-Net adapts best for every $K \geq 1$. A good initialisation matters most when the adaptation budget is small. With more steps the initialisation matters less, and the smaller network (7.76M instead of 18.55M parameters) adapts best from little data. The inner-loop adaptation does not improve with K, because 30 steps are too few to use more data.

5 Conclusion

We rebuilt the project from its first report, retrained every model and evaluated on the full-resolution SID test set and on held-out ELD scenes. The answers to the three questions are more nuanced than the first version suggested.

1. **Bottleneck refinement with FiLM helps; teacher-latent distillation does not.** The Bridge U-Net without distillation is the most accurate model (28.94 dB, +0.32 dB over our SID U-Net), and FiLM conditioning on the exposure ratio and noise level contributes 0.25 dB. Distillation towards the encoder's own long-exposure features constrains the bridge and costs 0.58 dB.

2. **A new camera needs very little data, and meta-learning helps only when adaptation is short.** One scene of an ELD camera adds about 2 dB over the zero-shot model, and five scenes match full fine-tuning. Once support and query are separated by scene, MAML gives the best starting point and the best result after a few inner steps. With a longer adaptation the plain SID U-Net is as good or better.

3. **A real diffusion bridge loses on fidelity metrics and wins on a perceptual one.** The feature-space variant of the first version is not a diffusion model and breaks under its own sampler. A simplified pixel-space DDBM reaches 28.03 dB in 5 steps, and more steps lower PSNR, as expected for posterior sampling. On LPIPS it beats the SID U-Net by 14% (0.394 vs. 0.459), the perception–distortion trade-off in action. The VE recipe of the released code trains far faster when its bridge noise is matched to the data range, but after 60k iterations and with 20 steps it stays below the simplified bridge on every metric. The full DDBM, with the released VP bridge and sampler and 303k iterations, is the clearest case of the trade-off: the best LPIPS (0.291, 37% below the U-Net) and the sharpest images, for 1.3 dB less PSNR, 0.10 less SSIM, about 300 times the run time and some invented detail. A faithful diffusion bridge therefore does not replace regression for low-light enhancement. It gives a different output, closer on a perceptual metric and further away pixel by pixel.

Limitations. Every configuration was trained once. A second run of the SID U-Net differs by only 0.01 dB, but two runs of one model are a weak variance estimate, so the smaller gaps in Table 2 (for example FiLM, 0.25 dB) need more seeds to be firm. The cloud machine running the first set of experiments was reclaimed before all files were copied. We kept every number reported here, but lost the checkpoints. The qualitative results and the LPIPS scores therefore cover only the four models trained afterwards (the SID U-Net and three pixel-space DDBMs), and the $\sigma_{\max} = 80$ DDBM has no test score. The full DDBM follows the released code in everything but the network, so our numbers do not cover the ADM U-Net with attention. LPIPS is one learned metric; a user study or a no-reference measure would make the perceptual comparison firmer.

Future work. The strongest single result is that one or a few scenes are enough to adapt to a new sensor. This makes on-device adaptation feasible: a phone could capture a handful of short/long RAW pairs of a static scene and fine-tune the decoder and bridge only. Combining this with calibrated noise synthesis [30], and distilling the diffusion bridge into one or two steps, are the natural next steps.

References

[1] Y. Blau and T. Michaeli. The perception-distortion tradeoff. In *CVPR*, 2018.

[2] C. Chen, Q. Chen, J. Xu, and V. Koltun. Learning to see in the dark. In *CVPR*, pages 3291–3300, 2018.

[3] K. Dabov, A. Foi, V. Katkovnik, and K. Egiazarian. Image denoising by sparse 3-d transform-domain collaborative filtering. *IEEE TIP*, 16(8):2080–2095, 2007.

[4] C. Finn, P. Abbeel, and S. Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *ICML*, 2017.

[5] C. Guo, C. Li, J. Guo, C. C. Loy, J. Hou, S. Kwong, and R. Cong. Zero-reference deep curve estimation for low-light image enhancement. In *CVPR*, 2020.

[6] S. W. Hasinoff, D. Sharlet, R. Geiss, A. Adams, J. T. Barron, F. Kainz, J. Chen, and M. Levoy. Burst photography for high dynamic range and low-light imaging on mobile cameras. *ACM TOG (SIGGRAPH Asia)*, 35(6), 2016.

[7] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv:1503.02531*, 2015.

[8] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.

[9] T. Hospedales, A. Antoniou, P. Micaelli, and A. Storkey. Meta-learning in neural networks: A survey. *IEEE TPAMI*, 44(9):5149–5169, 2022.

[10] Y. Jiang, X. Gong, D. Liu, Y. Cheng, C. Fang, X. Shen, J. Yang, P. Zhou, and Z. Wang. Enlightengan: Deep light enhancement without paired supervision. *IEEE TIP*, 30:2340–2349, 2021.

[11] T. Karras, M. Aittala, T. Aila, and S. Laine. Elucidating the design space of diffusion-based generative models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[12] E. H. Land. The retinex theory of color vision. *Scientific American*, 237(6):108–129, 1977.

[13] B. Li, K. Xue, B. Liu, and Y.-K. Lai. Bbdm: Image-to-image translation with brownian bridge diffusion models. In *CVPR*, 2023.

[14] G.-H. Liu, A. Vahdat, D.-A. Huang, E. A. Theodorou, W. Nie, and A. Anandkumar. I^2sb: Image-to-image schrödinger bridge. In *ICML*, 2023.

[15] A. Nichol, J. Achiam, and J. Schulman. On first-order meta-learning algorithms. *arXiv:1803.02999*, 2018.

[16] A. Parnami and M. Lee. Learning from few examples: A summary of approaches to few-shot learning. *arXiv:2203.04291*, 2022.

[17] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. Film: Visual reasoning with a general conditioning layer. In *AAAI*, 2018.

[18] K. R. Prabhakar, V. Vinod, N. R. Sahoo, and R. V. Babu. Few-shot domain adaptation for low light raw image enhancement. In *BMVC*, 2021.

[19] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio. Fitnets: Hints for thin deep nets. In *ICLR*, 2015.

[20] O. Ronneberger, P. Fischer, and T. Brox. U-net: Convolutional networks for biomedical image segmentation. In *MICCAI*, pages 234–241, 2015.

[21] C. Saharia, W. Chan, H. Chang, C. Lee, J. Ho, T. Salimans, D. Fleet, and M. Norouzi. Palette: Image-to-image diffusion models. In *SIGGRAPH*, 2022.

[22] C. Saharia, J. Ho, W. Chan, T. Salimans, D. J. Fleet, and M. Norouzi. Image super-resolution via iterative refinement. *IEEE TPAMI*, 2022.

[23] J. W. Soh, S. Cho, and N. I. Cho. Meta-transfer learning for zero-shot super-resolution. In *CVPR*, 2020.

[24] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. In *ICLR*, 2021.

[25] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. Score-based generative modeling through stochastic differential equations. In *ICLR*, 2021.

[26] V. Vapnik and A. Vashist. A new learning paradigm: Learning using privileged information. *Neural Networks*, 22(5-6):544–557, 2009.

[27] Z. Wang, E. P. Simoncelli, and A. C. Bovik. Multiscale structural similarity for image quality assessment. In *Asilomar Conference on Signals, Systems & Computers*, pages 1398–1402, 2003.

[28] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE TIP*, 13(4):600–612, 2004.

[29] C. Wei, W. Wang, W. Yang, and J. Liu. Deep retinex decomposition for low-light enhancement. In *BMVC*, 2018.

[30] K. Wei, Y. Fu, J. Yang, and H. Huang. A physics-based noise formation model for extreme low-light raw denoising. In *CVPR*, 2020.

[31] K. Wei, Y. Fu, Y. Zheng, and J. Yang. Physics-based noise modeling for extreme low-light photography. *IEEE TPAMI*, 2021.

[32] S. W. Zamir, A. Arora, S. Khan, M. Hayat, F. S. Khan, and M.-H. Yang. Restormer: Efficient transformer for high-resolution image restoration. In *CVPR*, 2022.

[33] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018.

[34] Y. Zhang, H. Qin, X. Wang, and H. Li. Rethinking noise synthesis and modeling in raw denoising. In *ICCV*, 2021.

[35] H. Zhao, O. Gallo, I. Frosio, and J. Kautz. Loss functions for image restoration with neural networks. *IEEE TCI*, 3(1):47–57, 2017.

[36] K. Zheng, G. He, J. Chen, F. Bao, and J. Zhu. Diffusion bridge implicit models. In *ICLR*, 2025.

[37] L. Zhou, A. Lou, S. Khanna, and S. Ermon. Denoising diffusion bridge models. In *ICLR*, 2024.